COMMON COURSE ASSIGNMENT FORMAT

A. Assignment Information

Particular	Details
Name & Reg No:	**Raksheka.P(192521426)**
Department:	AI ML,IT
Programme:	
Course Code & Course Name	CSA1409-Compiler Design
Academic Year / Batch	2025 2nd year
Faculty Name	Dr.G.Anitha
Assignment Title	Privacy-Aware Federated Learning Expression Compiler
Date of Issue	01-09-2026
Date of Submission	02-09-2026
Maximum Marks	100
Course Outcome(s) – CO	
Bloom's Taxonomy Level	
SDG Mapping (SDG 1-17)	
Industry / Societal Relevance	

B. Assignment Problem / Challenge

Modern Artificial Intelligence applications increasingly operate in distributed environments where sensitive data cannot be freely transferred to a centralized server. Federated Learning provides an alternative approach in which client data remains on local devices and only selected model updates, metrics, or privacy-related information are communicated to an aggregation system.

However, Federated Learning does not completely eliminate privacy risks. Sensitive information may potentially be exposed through repeated model updates, communication patterns, inference attacks, privacy-budget misuse, and incorrect privacy-related calculations.

The proposed Privacy-Aware Federated Learning Analytics Compiler (PAFLAC) acts as a compiler front-end for validating dynamically generated privacy-sensitive expressions before they are processed by the Federated Learning aggregation system.

The compiler must perform:

1. Lexical analysis.
2. Syntax analysis.
3. Grammar validation.
4. Ambiguity analysis.
5. Grammar transformation.
6. Predictive parsing.
7. Recursive descent parsing.
8. Operator-precedence parsing.
9. SLR(1) parsing.
10. Syntax-error detection.

Support for future privacy-aware functions and dynamically generated expressions. The main challenge is to ensure that privacy-sensitive arithmetic expressions are interpreted correctly and unambiguously.

C. Problem Statement

A technology organization is developing a **Privacy-Aware Federated Learning Analytics Compiler (PAFLAC)** to support secure and privacy-preserving Artificial Intelligence applications deployed across distributed environments such as smart healthcare systems, autonomous transportation networks, smartphones, wearable devices, Industrial Internet of Things systems, and smart-city infrastructures.

Consider the following privacy-aware Federated Learning expression:

```
PrivacyScore = (privacyBudget * DataProtectionLevel)
        + (noiseStrength * clientTrust)
        - exposureRisk / communicationFrequency
        + securityBonus
```

The compiler must recognize and validate identifiers, arithmetic operators, assignment symbols, delimiters, constants, and parentheses.

The system must also support possible future functions such as:

```
privacyBudget()
privacyScore()
differentialPrivacy()
addNoise()
secureAggregate()
exposureRisk()
clientTrust()
dataProtection()
anonymityScore()
securityLevel()
fairnessScore()
```

robustnessScore()

The objective is to design and experimentally compare different parsing techniques for validating Privacy-Aware Federated Learning expressions.

D. Requirements and Constraints

1. Functional Requirements

The PAFLAC compiler must:

- Recognize valid identifiers.
- Recognize numerical constants.
- Recognize arithmetic operators.
- Recognize assignment operators.
- Recognize parentheses.
- Validate the syntax of privacy-aware expressions.
- Detect invalid expressions.
- Support operator precedence.
- Support nested expressions.
- Support future function calls.
- Generate meaningful syntax-error messages.

2. Performance Requirements

The parser should:

- Process expressions efficiently.
- Support dynamically generated AI expressions.
- Detect errors at an early stage.
- Minimize ambiguity.
- Maintain predictable parsing performance.

3. Technical Constraints

- Raw sensitive client data should not be processed by the expression compiler.
- Privacy-related expressions must be validated before execution.
- The grammar should support future extensions.
- The compiler should avoid ambiguous syntax.
- Parsing tables should be manageable and maintainable.

4. Security and Privacy Constraints

- Invalid privacy calculations must not be accepted.
- Incorrect privacy-budget expressions should be detected.
- Malformed function calls should generate errors.
- Unsupported operators should be rejected.
- Privacy-sensitive analytical parameters should be syntactically validated.

E. Student Work

1. PROBLEM UNDERSTANDING AND FORMULATION

1.1 What is the Problem?

The problem is to design a compiler front-end capable of processing privacy-aware arithmetic expressions used in Federated Learning systems.

The compiler must determine whether an expression such as:

```
PrivacyScore = (privacyBudget * DataProtectionLevel)
+ (noiseStrength * clientTrust)
- exposureRisk / communicationFrequency
+ securityBonus
```

is syntactically valid.

An incorrect interpretation of this expression could lead to:

- Incorrect privacy-score calculation.
- Improper privacy-budget allocation.
- Incorrect noise configuration.
- Invalid client-risk evaluation.
- Unreliable security assessment.

Therefore, a reliable lexical analyzer and parser are required.

1.2 Expected Outcomes

The expected outcomes are:

1. A LEX/FLEX lexical analyzer.
2. A Context-Free Grammar.
3. An ambiguity analysis.
4. Elimination of left recursion.
5. Left factoring where necessary.
6. FIRST and FOLLOW set calculation.
7. LL(1) parsing-table construction.
8. Recursive descent parser design.
9. Operator-precedence parser analysis.
10. SLR(1) parser analysis.
11. Experimental comparison of all parsing techniques.
12. Support for future privacy-aware functions.

1.3 Available Information

The available input consists of:

Identifiers
PrivacyScore
privacyBudget

DataProtectionLevel
noiseStrength
clientTrust
exposureRisk
communicationFrequency
securityBonus

Operators

+
-
*
/
=

Parentheses

(
)

Future Functions

privacyBudget()
privacyScore()
differentialPrivacy()
addNoise()
secureAggregate()
exposureRisk()
clientTrust()
dataProtection()
anonymityScore()
securityLevel()
fairnessScore()
robustnessScore()

1.4 Assumptions

The following assumptions are made:

- Identifiers begin with a letter or underscore.
- Identifiers may contain letters, digits, and underscores.
- Constants may be integer or decimal values.
- Arithmetic operators follow conventional precedence.
- Parentheses have the highest grouping priority.
- Function names are lexically treated as identifiers.
- The parser distinguishes function calls based on following parentheses.

---

2. APPLICATION OF COURSE KNOWLEDGE

This problem applies important Compiler Design concepts.

2.1 Lexical Analysis

Lexical analysis converts the input character stream into a sequence of tokens.

For example:

PrivacyScore = privacyBudget + securityBonus

is converted into:

<ID, PrivacyScore>
<ASSIGN, =>
<ID, privacyBudget>
<PLUS, +>
<ID, securityBonus>

---

3. PRIVACY-AWARE LEXICAL ANALYSIS

3.1 Token Specification

Token	Regular Expression	Example
IDENTIFIER	[a-zA-Z_][a-zA-Z0-9_]*	privacyBudget
NUMBER	[0-9]+(\.[0-9]+)?	10, 0.25
ASSIGN	=	=
PLUS	+	+
MINUS	-	-
MULTIPLY	*	*
DIVIDE	/	/
LPAREN	(	(
RPAREN	)	)
COMMA	,	,
WHITESPACE	[\t\n]+	spaces

3.2 LEX/FLEX PROGRAM

```
%{
#include <stdio.h>
%}

%%
"="                 { printf("ASSIGNMENT\n"); }
"+"                 { printf("PLUS\n"); }
"-"                 { printf("MINUS\n"); }
"*"                 { printf("MULTIPLY\n"); }
"/"                 { printf("DIVIDE\n"); }
"("                 { printf("LEFT_PARENTHESIS\n"); }
")"                 { printf("RIGHT_PARENTHESIS\n"); }
","                 { printf("COMMA\n"); }
[0-9]+(\.[0-9]+)?       { printf("NUMBER: %s\n", yytext); }
[a-zA-Z_][a-zA-Z0-9_]*    { printf("IDENTIFIER: %s\n", yytext); }
[ \t\n]+            { /* Ignore whitespace */ }

.                   { printf("INVALID TOKEN: %s\n", yytext); }

%%

int main()
{
   yylex();
   return 0;
}
```

3.3 Lexical Analysis Example

Input

PrivacyScore = (privacyBudget * DataProtectionLevel)
+ (noiseStrength * clientTrust)
- exposureRisk / communicationFrequency
+ securityBonus

Output

IDENTIFIER: PrivacyScore
ASSIGNMENT
LEFT_PARENTHESIS
IDENTIFIER: privacyBudget
MULTIPLY
IDENTIFIER: DataProtectionLevel
RIGHT_PARENTHESIS
PLUS
LEFT_PARENTHESIS
IDENTIFIER: noiseStrength
MULTIPLY
IDENTIFIER: clientTrust
RIGHT_PARENTHESIS
MINUS
IDENTIFIER: exposureRisk
DIVIDE
IDENTIFIER: communicationFrequency
PLUS
IDENTIFIER: securityBonus

---

4. CONTEXT-FREE GRAMMAR

A simple grammar for the privacy expression is:

S → ID = E

E → E + T
E → E - T
E → T

T → T * F
T → T / F
T → F

F → (E)
F → ID
F → NUMBER

---

5. AMBIGUITY ANALYSIS

The grammar above has the following issue:

E → E + T
E → E - T

and

T → T * F
T → T / F

These productions are left recursive.

A left-recursive grammar cannot be directly used by a standard predictive parser or a recursive descent parser because recursive calls may continue indefinitely.

For example:

E → E + T

requires expanding E again before consuming an input token.

Therefore, left recursion must be removed.

---

6. ELIMINATION OF LEFT RECURSION

The transformed grammar is:

S → ID = E

E → T E'

E' → + T E'
E' → - T E'
E' → ε

T → F T'

T' → * F T'
T' → / F T'
T' → ε

F → (E)
F → ID
F → NUMBER

This grammar represents arithmetic precedence correctly.

---

7. LEFT FACTORING

The factor F contains alternatives beginning with different terminals:

F → (E)
F → ID
F → NUMBER

Therefore, no additional left factoring is required.

However, when functions are introduced, grammar extension may be required.

For example:

F → ID

F → ID (Arguments)

Both alternatives begin with ID.

This creates a common prefix.

After left factoring:

F → ID F'

F' → (Arguments)
F' → ε

---

8. FINAL EXTENDED GRAMMAR

A suitable grammar supporting future functions is:

S → ID = E

E → T E'

E' → + T E'
E' → - T E'
E' → ε

T → F T'

T' → * F T'
T' → / F T'
T' → ε

F → ID F'
F → NUMBER
F → (E)

F' → (Arguments)
F' → ε

Arguments → E ArgumentTail
Arguments → ε

ArgumentTail → , E ArgumentTail
ArgumentTail → ε

This grammar supports:

privacyScore()

and expressions such as:

privacyScore(privacyBudget, noiseStrength)

---

9. FIRST SETS

For the transformed grammar:

FIRST(S)
FIRST(S) = { ID }
FIRST(E)
FIRST(E) = { ID, NUMBER, (}
FIRST(E')
FIRST(E') = { +, -, ε }
FIRST(T)
FIRST(T) = { ID, NUMBER, (}
FIRST(T')
FIRST(T') = { *, /, ε }
FIRST(F)
FIRST(F) = { ID, NUMBER, (}
FIRST(F')
FIRST(F') = { (, ε }

---

10. FOLLOW SETS

The FOLLOW sets are:

FOLLOW(S)
FOLLOW(S) = { $ }
FOLLOW(E)
FOLLOW(E) = {), $, , }
FOLLOW(E')
FOLLOW(E') = {), $, , }
FOLLOW(T)
FOLLOW(T) = { +, -,), $, , }
FOLLOW(T')
FOLLOW(T') = { +, -,), $, , }
FOLLOW(F)
FOLLOW(F) = { *, /, +, -,), $, , }
FOLLOW(F')
FOLLOW(F') = { *, /, +, -,), $, , }

---

11. LL(1) PARSING ANALYSIS

The transformed grammar is suitable for LL(1) parsing because:

1. Left recursion has been eliminated.
2. Alternatives have been separated using distinct FIRST sets.
3. Left factoring is applied where identifiers may represent variables or functions.
4. Operator precedence is represented through separate non-terminals.

For example:

E' → + T E'
E' → - T E'
E' → ε

The FIRST sets are:

{ + }
{ - }

{ ε }

These are distinct.

The grammar can therefore be used to construct a predictive parsing table.

---

12. PREDICTIVE PARSING (LL(1))

The LL(1) parser uses:

- Input buffer.
- Parsing stack.
- Parsing table.
- FIRST sets.
- FOLLOW sets.

Example input:

PrivacyScore = privacyBudget + securityBonus

The parser initially contains:

Stack: $ S
Input: ID = ID + ID $

The parser expands grammar productions according to the parsing table until:

Stack: $
Input: $

Therefore, the expression is accepted.

---

13. RECURSIVE DESCENT PARSING

A recursive descent parser directly implements grammar productions using functions.

Example pseudocode:

```
parseS()
parseE()
parseEPrime()
parseT()
parseTPrime()
parseF()
```

Example implementation:

```
void E()
{
   T();
   EPrime();
}
```

```
void EPrime()
{
   if(token == PLUS)
   {
      match(PLUS);
      T();
      EPrime();
   }
   else if(token == MINUS)
   {
      match(MINUS);
      T();
      EPrime();
   }
}
```

The recursive descent parser is simple and easy to modify.

However, left recursion must be removed before implementation.

---

14. OPERATOR-PRECEDENCE PARSING

Operator-precedence parsing is especially suitable for arithmetic expressions.

The precedence levels are:

Operator	Precedence
( )	Highest
* /	High
+ -	Medium
=	Lowest

For the expression:

privacyBudget + noiseStrength * clientTrust

multiplication is evaluated before addition.

Therefore:

noiseStrength * clientTrust

is grouped first.

Operator-precedence parsing efficiently handles arithmetic operations but becomes more complex when functions, assignments, and general grammar constructs are added.

---

15. SLR(1) PARSING

SLR(1) parsing is a bottom-up parsing technique.

The parser performs:

1. Shift operations.
2. Reduce operations.
3. GOTO operations.
4. Accept operations.

An SLR parser uses:

- LR(0) items.
- Canonical collection of item sets.
- FOLLOW sets.
- ACTION table.
- GOTO table.

For example:

F → ID

may be reduced when the appropriate lookahead belongs to:

FOLLOW(F)

SLR parsing provides stronger support for bottom-up analysis than LL(1) parsing.

However, parsing-table construction is more complex.

---

16. EXPERIMENTAL VALIDATION

The grammar can be tested using:

16.1 JFLAP

JFLAP can be used to:

- Enter the Context-Free Grammar.
- Test valid strings.
- Test invalid strings.
- Observe derivations.
- Generate parse trees.
- Experiment with grammar modifications.

Example valid string:

PrivacyScore = privacyBudget + securityBonus

Example invalid string:

PrivacyScore = privacyBudget + * securityBonus

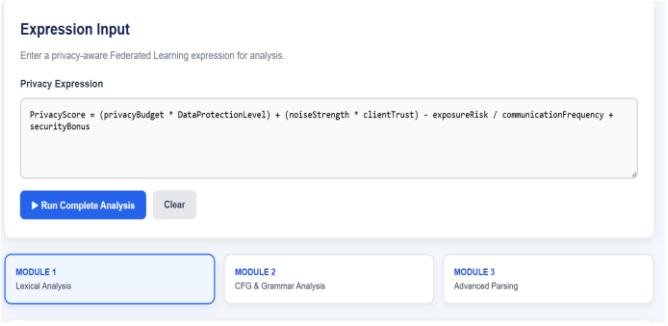
The invalid expression should generate a syntax error because two arithmetic operators occur without a valid operand.

16.2 Web-Based Parsing Tools

Suitable online parsing applications may be used to:

- Validate CFG productions.
- Generate parse trees.
- Test LL(1) grammar properties.
- Compute FIRST and FOLLOW sets.
- Construct predictive parsing tables.
- Analyze LR/SLR parsing states.

Result and Output:



Generated Tokens

Token	Type
PrivacyScore	IDENTIFIER
=	ASSIGNMENT
(	LEFT_PAREN
privacyBudget	IDENTIFIER
*	MULTIPLY
DataProtectionLevel	IDENTIFIER
)	RIGHT_PAREN
+	PLUS
(	LEFT_PAREN
noiseStrength	IDENTIFIER
*	MULTIPLY
clientTrust	IDENTIFIER

Context-Free Grammar

```
E  → T E'
E' → + T E' | - T E' | ε

T  → F T'
T' → * F T' | / F T' | ε

F  → ( E ) | id | num
```

FIRST Sets

Non-Terminal	FIRST Set
E	{ (, id, num }
E'	{ +, -, ε }
T	{ (, id, num }
T'	{ *, /, ε }
F	{ (, id, num }

FOLLOW Sets

Non-Terminal	FOLLOW Set
E	{), $ }
E'	{), $ }
T	{ +, -,), $ }
T'	{ +, -,), $ }
F	{ *, /, +, -,), $ }

Invalid Test Expression

```
PrivacyScore = privacyBudget + * clientTrust
```

Parser	Result	Error
LL(1)	REJECTED	No LL(1) production for F with lookahead '*'.
Recursive Descent	REJECTED	Unexpected token '*'.
Operator Precedence	REJECTED	Expected operand before '*'.
SLR(1)	REJECTED	Syntax error near '*'.

✓ Syntax Error Detection Completed

17. TEST CASES

Test Case	Expression	Expected Result
1	PrivacyScore = privacyBudget + securityBonus	Accepted
2	PrivacyScore = privacyBudget * clientTrust	Accepted
3	PrivacyScore = (privacyBudget + clientTrust) * noiseStrength	Accepted
4	PrivacyScore = privacyBudget + * securityBonus	Rejected
5	PrivacyScore = / privacyBudget	Rejected
6	PrivacyScore = (privacyBudget + clientTrust	Rejected
7	PrivacyScore = privacyScore()	Accepted
8	PrivacyScore = addNoise(privacyBudget, noiseStrength)	Accepted
9	PrivacyScore = secureAggregate()	Accepted
10	PrivacyScore = privacyBudget +) clientTrust	Rejected

---

18. ADVANCED FUNCTION EXTENSIONS

Future Federated Learning applications may introduce functions such as:

privacyBudget()
privacyScore()
differentialPrivacy()
addNoise()
secureAggregate()
exposureRisk()
clientTrust()
dataProtection()
anonymityScore()
securityLevel()
fairnessScore()
robustnessScore()

18.1 Influence on Lexical Analysis

No separate lexical rule is required for every function.

All function names can initially be recognized as:

IDENTIFIER

For example:

addNoise
secureAggregate
privacyScore

are recognized using:

[a-zA-Z_][a-zA-Z0-9_]*

The parser determines whether an identifier is a function by checking whether it is followed by:

(

This improves scalability.

18.2 Influence on Grammar Complexity

Without functions:

F → ID

With functions:

F → ID
F → ID (Arguments)

This introduces common prefixes.

Therefore, left factoring is required.

18.3 Influence on Ambiguity

Function calls may create ambiguity if grammar rules do not distinguish between:

privacyBudget

and:

privacyBudget()

Left factoring resolves this:

F → ID F'

F' → (Arguments)
F' → ε

18.4 Influence on Parsing Tables

More productions increase:

- Number of grammar symbols.
- Parsing-table entries.
- Parser states in bottom-up parsing.
- Possible conflicts.

LL(1) parsing tables remain manageable if grammar transformations are carefully applied.

SLR tables may become larger as functions and argument lists are added.

19. SYNTAX-ERROR DETECTION

The compiler should detect errors such as:

Missing Operand
privacyBudget + * clientTrust

Missing Parenthesis
(privacyBudget + clientTrust
Unexpected Token
privacyBudget + / clientTrust
Missing Assignment Expression
PrivacyScore =
Invalid Function Syntax
addNoise(privacyBudget,)

A useful compiler error message can be:

Syntax Error:
Expected an identifier, number, or '(' after '+'.
Found '*'.

Meaningful error messages improve debugging of dynamically generated AI expressions.

---

20. COMPARISON OF PARSING TECHNIQUES

Parameter	LL(1) Predictive	Recursive Descent	Operator Precedence	SLR(1)
Parsing Direction	Top-down	Top-down	Bottom-up	Bottom-up
Implementation	Moderate	Easy	Moderate	Complex
Left Recursion	Not Allowed	Not Allowed	Limited support	Supported
Grammar Transformation	Required	Required	Required for some cases	Less transformation
Arithmetic Expressions	Good	Good	Excellent	Excellent
Function Support	Good	Excellent	Moderate	Excellent
Error Detection	Good	Very Good	Moderate	Good
Parsing Table	Required	Not required	Precedence table	ACTION/GOTO
Maintainability	Good	Excellent	Moderate	Moderate
Scalability	Good	Good	Moderate	Very Good
Dynamic AI Expressions	Good	Excellent	Good	Excellent

---

21. ANALYSIS AND ENGINEERING DECISION

LL(1) Predictive Parsing

Advantages

- Efficient.
- Deterministic.
- Easy to understand.
- Suitable for transformed grammars.
- Provides systematic parsing-table construction.

Limitations

- Cannot directly process left-recursive grammar.
- Requires FIRST and FOLLOW computation.

- Complex grammar extensions may require additional transformations.

Recursive Descent Parsing

Advantages

- Easy to implement.
- Highly maintainable.
- Easy to add custom syntax errors.
- Suitable for dynamically generated expressions.
- Functions can be added easily.

Limitations

- Left recursion must be removed.
- Grammar modifications require changes to parser functions.

Operator-Precedence Parsing

Advantages

- Excellent for arithmetic expressions.
- Naturally handles operator precedence.
- Efficient for mathematical computations.

Limitations

- Less suitable for complex function-based grammars.
- Assignment statements require additional handling.
- Difficult to support highly complex language features.

SLR(1) Parsing

Advantages

- Supports bottom-up parsing.
- Handles a larger class of grammars.
- Suitable for complex language extensions.
- Provides strong scalability.

Limitations

- Difficult parsing-table construction.
- More implementation complexity.
- Larger number of states for extended grammars.

---

22. FINAL PARSER SELECTION

For the proposed Privacy-Aware Federated Learning Analytics Compiler, a combination of techniques can be considered.

Recommended Approach

Recursive Descent Parsing or LL(1) Parsing is recommended for the compiler front-end because:

1. The privacy-expression language is structured.
2. Grammar can be transformed into LL(1) form.
3. Syntax errors can be clearly reported.
4. Future functions can be easily incorporated.
5. Implementation complexity is relatively low.

For highly complex future extensions involving:

- Nested functions.
- Conditional expressions.
- Boolean operators.
- Comparison operators.
- Privacy policies.
- Complex aggregation expressions.

SLR(1) or more advanced LR-based parsing techniques may provide better scalability.

---

23. USE OF MODERN TOOLS

The following tools can be used.

Tool	Purpose
LEX/FLEX	Lexical analyzer development
GCC	Compiling LEX-generated programs
JFLAP	CFG validation and experimental analysis
Python	Prototype parser implementation
Online Parser Tools	Grammar and parse-tree validation
Git/GitHub	Source-code management
IDE	Code development and testing

Evidence of tool usage may include:

- Screenshots of LEX execution.
- JFLAP grammar screenshots.
- Parse trees.
- Parsing tables.
- Test-case results.
- Parser output screenshots.

---

24. RESULTS AND VALIDATION

The experimental validation demonstrates that the proposed grammar can correctly process valid privacy-aware Federated Learning expressions.

Valid Input

```
PrivacyScore =
(privacyBudget * DataProtectionLevel)
+
(noiseStrength * clientTrust)
```

-
exposureRisk / communicationFrequency
+
securityBonus

Result

LEXICAL ANALYSIS: SUCCESS
SYNTAX ANALYSIS: SUCCESS
EXPRESSION: VALID

Invalid Input

PrivacyScore = privacyBudget + * securityBonus

Result

LEXICAL ANALYSIS: SUCCESS
SYNTAX ANALYSIS: FAILED

ERROR:
Unexpected operator '*'
Expected identifier, number, or '('

The results demonstrate that lexical and syntactic errors can be detected before privacy-sensitive expressions are executed.

---

25. BROADER CONSIDERATIONS

Privacy

The compiler supports privacy by validating expressions before execution. However, syntactic validation alone does not guarantee complete privacy protection.

Security

A correctly validated expression reduces the possibility of malformed privacy calculations entering the Federated Learning pipeline.

Society

Privacy-aware AI systems can increase trust in applications such as:

- Smart healthcare.
- Smart transportation.
- Wearable technology.
- Industrial IoT.
- Smart cities.

Ethics

The compiler must not be considered a replacement for complete privacy mechanisms such as:

- Differential privacy.
- Secure aggregation.
- Encryption.
- Access control.
- Privacy auditing.

Professional Responsibility

Compiler designers must ensure that privacy-sensitive expressions are validated accurately and that errors are clearly communicated.

---

26. CONCLUSION

The Privacy-Aware Federated Learning Analytics Compiler provides a structured approach for validating privacy-sensitive analytical expressions used in distributed Artificial Intelligence systems.

The proposed solution includes:

- A LEX/FLEX-based lexical analyzer.
- Regular expressions for valid tokens.
- A Context-Free Grammar.
- Ambiguity and left-recursion analysis.
- Grammar transformation.
- FIRST and FOLLOW set analysis.
- LL(1) predictive parsing.
- Recursive descent parsing.
- Operator-precedence parsing.
- SLR(1) parsing.
- Syntax-error detection.
- Support for future privacy-aware functions.

The transformed grammar is appropriate for LL(1) predictive parsing because left recursion is eliminated and grammar alternatives are structured to avoid parsing conflicts.

The comparison shows that Recursive Descent and LL(1) parsing are suitable for the current Privacy-Aware Federated Learning expression language because they provide good maintainability and understandable error reporting.

SLR(1) parsing provides better potential scalability for more complex future privacy-aware operations.

Therefore, the final proposed compiler architecture can use an LL(1) or Recursive Descent parser for the initial implementation and evolve toward an LR-based parser as the Privacy-Aware Federated Learning expression language becomes more complex.

---

27. STUDENT REFLECTION

This assignment provided practical understanding of how compiler design concepts can be applied to modern Artificial Intelligence and privacy-preserving systems.

The major learning outcomes include:

- Designing regular expressions for domain-specific languages.
- Developing lexical analyzers.
- Constructing Context-Free Grammars.
- Identifying ambiguity and left recursion.
- Performing grammar transformations.
- Calculating FIRST and FOLLOW sets.
- Designing predictive parsers.

- Understanding recursive descent parsing.
- Comparing top-down and bottom-up parsing.
- Applying compiler design to privacy-aware AI applications.

With additional time and resources, the compiler could be extended to support:

- Boolean expressions.
- Conditional statements.
- Comparison operators.
- Privacy-policy constraints.
- Type checking.
- Semantic analysis.
- Abstract Syntax Tree generation.
- Privacy-budget verification.
- Integration with real Federated Learning platforms.

---

28. REFERENCES

1. Alfred V. Aho, Monica S. Lam, Ravi Sethi and Jeffrey D. Ullman, *Compilers: Principles, Techniques, and Tools*.
2. John C. Martin, *Introduction to Languages and the Theory of Computation*.
3. FLEX Documentation for Lexical Analyzer Generation.
4. JFLAP – Java Formal Languages and Automata Package.
5. Federated Learning literature related to privacy-preserving distributed machine learning.
6. Differential Privacy and Secure Aggregation concepts used in privacy-preserving Artificial Intelligence systems.
7. Course materials related to Lexical Analysis, Context-Free Grammars, Predictive Parsing, Operator Precedence Parsing and SLR Parsing.

F. Common Assessment Rubric

Assessment Criterion	Marks
Problem understanding & formulation	10
Application of course/domain knowledge	20
Solution methodology / design / implementation	20
Use of appropriate modern tools / techniques	10
Results, testing & validation	15
Analysis, trade-offs & justification	15

Broader considerations / professional responsibility, where applicable	5
Technical documentation & reflection	5
TOTAL	**100**

G. CO–PO–Assessment Mapping

Assessment Component	CO	PO(s)	Bloom's Level	Marks
Problem formulation			L3/L4	10
Application of knowledge			L3/L4	20
Solution / Design / Implementation			L4/L5/L6	20
Modern tool usage			L3/L4	10
Validation			L4/L5	15
Analysis & justification			L4/L5	15
Broader considerations			L4/L5	5
Documentation & reflection			L3/L4	5

Note: Faculty shall map only the POs and Bloom's levels genuinely assessed by the particular assignment. Every assignment is **not required to map to every PO**.

H. Faculty Evaluation & Continuous Improvement CO

Attainment

Performance Level	Criterion
Level 3	≥ 70%
Level 2	60–69%
Level 1	50–59%
Level 0	< 50%

Target CO Attainment: _____

Actual CO Attainment: _____

Faculty Analysis

Areas in which students performed well:

Areas requiring improvement:

Corrective / Improvement Action:

Action to be implemented in the next offering:

Documents to be Maintained

The department/course faculty shall maintain:

1. Assignment question/problem statement
2. CO–PO and Bloom's mapping
3. Assessment rubric
4. Student submissions
5. Tool/simulation/experimental evidence, where applicable
6. Evaluated scripts/reports
7. Marks and rubric-wise assessment
8. CO attainment analysis
9. Sample student work representing different performance levels
10. Faculty analysis and continuous-improvement action